```
medical_text = """\nA patient presented with severe abdominal pain. Doctors worked to diagnose the

print("Medical text corpus created successfully.")
```

```
Medical text corpus created successfully.
```

```
import nltk
nltk.download('punkt', quiet=True)
print("NLTK 'punkt' tokenizer data downloaded.")
```

```
NLTK 'punkt' tokenizer data downloaded.
```

```
nltk_sentences = nltk.sent_tokenize(medical_text)
nltk_words = []
for sent in nltk_sentences:
    nltk_words.extend(nltk.word_tokenize(sent))

print("NLTK Sentence Tokenization Result (first 3 sentences):")
for i, sent in enumerate(nltk_sentences[:3]):
    print(f"  {i+1}. {sent}")
print(f"\nTotal NLTK sentences: {len(nltk_sentences)}")
print("\nNLTK Word Tokenization Result (first 20 words):")
print(nltk_words[:20])
print(f"Total NLTK words: {len(nltk_words)}")
```

```
NLTK Sentence Tokenization Result (first 3 sentences):
  1.
A patient presented with severe abdominal pain.
  2. Doctors worked to diagnose the condition.
  3. After extensive tests, a diagnosis of appendicitis was made.

Total NLTK sentences: 12

NLTK Word Tokenization Result (first 20 words):
['A', 'patient', 'presented', 'with', 'severe', 'abdominal', 'pain', '.', 'Doctors', 'worked', 'to
Total NLTK words: 108
```

```
import nltk
nltk.download('punkt_tab', quiet=True)
print("NLTK 'punkt_tab' tokenizer data downloaded.")
```

```
NLTK 'punkt_tab' tokenizer data downloaded.
```

```
nltk_sentences = nltk.sent_tokenize(medical_text)
nltk_words = []
for sent in nltk_sentences:
    nltk_words.extend(nltk.word_tokenize(sent))

print("NLTK Sentence Tokenization Result (first 3 sentences):")
for i, sent in enumerate(nltk_sentences[:3]):
    print(f"  {i+1}. {sent}")
print(f"\nTotal NLTK sentences: {len(nltk_sentences)}")
print("\nNLTK Word Tokenization Result (first 20 words):")
print(nltk_words[:20])
print(f"Total NLTK words: {len(nltk_words)}")
```

```
NLTK Sentence Tokenization Result (first 3 sentences):
  1.
A patient presented with severe abdominal pain.
  2. Doctors worked to diagnose the condition.
  3. After extensive tests, a diagnosis of appendicitis was made.

Total NLTK sentences: 12

NLTK Word Tokenization Result (first 20 words):
['A', 'patient', 'presented', 'with', 'severe', 'abdominal', 'pain', '.', 'Doctors', 'worked', 'to
Total NLTK words: 108
```

```
import spacy
try:
    nlp = spacy.load("en_core_web_sm")
    print("spaCy 'en_core_web_sm' model loaded successfully.")
except OSError:
    print("Downloading spaCy 'en_core_web_sm' model...")
    !python -m spacy download en_core_web_sm
    nlp = spacy.load("en_core_web_sm")
    print("spaCy 'en_core_web_sm' model downloaded and loaded.")

spacy_doc = nlp(medical_text)
print("Medical text processed with spaCy.")
```

```
spaCy 'en_core_web_sm' model loaded successfully.
Medical text processed with spaCy.
```

```
spacy_sentences = [sent.text for sent in spacy_doc.sents]

print("spaCy Sentence Tokenization Result (first 3 sentences):")
for i, sent in enumerate(spacy_sentences[:3]):
    print(f"  {i+1}. {sent}")
print(f"\nTotal spaCy sentences: {len(spacy_sentences)}")
```

```
spaCy Sentence Tokenization Result (first 3 sentences):
  1.
A patient presented with severe abdominal pain.
  2. Doctors worked to diagnose the condition.

  3. After extensive tests, a diagnosis of appendicitis was made.

Total spaCy sentences: 12
```

```
spacy_words = [token.text for token in spacy_doc]

print("spaCy Word Tokenization Result (first 20 words):")
print(spacy_words[:20])
print(f"Total spaCy words: {len(spacy_words)}")
```

```
spaCy Word Tokenization Result (first 20 words):
['\n', 'A', 'patient', 'presented', 'with', 'severe', 'abdominal', 'pain', '.', 'Doctors', 'worked
Total spaCy words: 118
```

```
import nltk
nltk.download('wordnet', quiet=True)
print("NLTK 'wordnet' corpus downloaded.")
```

```
NLTK 'wordnet' corpus downloaded.
```

```
from nltk.stem import PorterStemmer, WordNetLemmatizer

# 1. Instantiate PorterStemmer
porter_stemmer = PorterStemmer()

# 2. Apply NLTK Porter Stemmer
nltk_stemmed_words = [porter_stemmer.stem(word) for word in nltk_words]

# 5. Instantiate WordNetLemmatizer
wordnet_lemmatizer = WordNetLemmatizer()

# 6. Apply NLTK WordNet Lemmatizer
nltk_lemmatized_words = [wordnet_lemmatizer.lemmatize(word) for word in nltk_words]

# 7. Apply spaCy Lemmatization
spacy_lemmatized_words = [token.lemma_.lower() for token in spacy_doc if token.is_alpha]

# 8. Print the first 20 stemmed/lemmatized words
print("\n--- Stemming and Lemmatization Results ---")
print("\nNLTK Porter Stemmed Words (first 20):")
print(nltk_stemmed_words[:20])
print(f"Total NLTK Stemmed Words: {len(nltk_stemmed_words)}")

print("\nNLTK WordNet Lemmatized Words (first 20):")
print(nltk_lemmatized_words[:20])
print(f"Total NLTK Lemmatized Words: {len(nltk_lemmatized_words)}")

print("\nspaCy Lemmatized Words (first 20):")
print(spacy_lemmatized_words[:20])
print(f"Total spaCy Lemmatized Words: {len(spacy_lemmatized_words)}")
```

```
--- Stemming and Lemmatization Results ---

NLTK Porter Stemmed Words (first 20):
['a', 'patient', 'present', 'with', 'sever', 'abdomin', 'pain', '.', 'doctor', 'work', 'to', 'diag
Total NLTK Stemmed Words: 108

NLTK WordNet Lemmatized Words (first 20):
['A', 'patient', 'presented', 'with', 'severe', 'abdominal', 'pain', '.', 'Doctors', 'worked', 'to
Total NLTK Lemmatized Words: 108

spaCy Lemmatized Words (first 20):
['a', 'patient', 'present', 'with', 'severe', 'abdominal', 'pain', 'doctor', 'work', 'to', 'diagno
Total spaCy Lemmatized Words: 94
```

```
import pandas as pd

example_words = ['presented', 'diagnosing', 'treatment', 'cells', 'Doctors', 'worked', 'analyzed',

comparison_data = []

for word in example_words:
    original_word = word
    # Find corresponding stemmed word (case-insensitive for comparison)
    nltk_stemmed = ''
    for i, nw in enumerate(nltk_words):
        if nw.lower() == original_word.lower():
            nltk_stemmed = nltk_stemmed_words[i]
            break
```

```
            break

    # Find corresponding NLTK lemmatized word (case-insensitive for comparison)
    nltk_lemmatized = ''
    for i, nw in enumerate(nltk_words):
        if nw.lower() == original_word.lower():
            nltk_lemmatized = nltk_lemmatized_words[i]
            break

    # Find corresponding spaCy lemmatized word (case-insensitive for comparison)
    spacy_lemmatized = ''
    # Iterate through spacy_doc to find the token and its lemma
    for token in spacy_doc:
        if token.text.lower() == original_word.lower():
            spacy_lemmatized = token.lemma_.lower()
            break

    comparison_data.append({
        'Original Word': original_word,
        'NLTK Stemmed': nltk_stemmed,
        'NLTK Lemmatized': nltk_lemmatized,
        'spaCy Lemmatized': spacy_lemmatized
    })

comparison_df = pd.DataFrame(comparison_data)

print("Comparison of Stemming and Lemmatization for Example Words:")
print(comparison_df.to_string(index=False))
```

```
Comparison of Stemming and Lemmatization for Example Words:
Original Word NLTK Stemmed NLTK Lemmatized spaCy Lemmatized
    presented      present       presented          present
   diagnosing      diagnos      diagnosing         diagnose
    treatment    treatment       treatment        treatment
        cells         cell            cell             cell
      Doctors       doctor         Doctors           doctor
       worked         work          worked             work
     analyzed       analyz        analyzed          analyze
     diseases       diseas         disease          disease
     outcomes       outcom         outcome          outcome
```